

Spring Testing Exercises

Exercise 1: Basic Unit Test for a Service Method

CalculatorServiceTest.java

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorServiceTest {

    CalculatorService calculatorService = new CalculatorService();

    @Test
    public void testAdd() {
        int result = calculatorService.add(4, 6);
        assertEquals(10, result);
    }
}
```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 2: Mocking a Repository in a Service Test

UserServiceTest.java

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
public class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    @Test
    public void testGetUserById() {
        User mockUser = new User();
    }
```

```

        mockUser.setId(1L);
        mockUser.setName("Ravi Kumar");

        when(userRepository.findById(1L)).thenReturn(Optional.of(mockUser));

        User result = userService.getUserById(1L);

        assertNotNull(result);
        assertEquals("Ravi Kumar", result.getName());
    }
}

```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 3: Testing a REST Controller with MockMvc

UserControllerTest.java

```

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.Mockito.*;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(UserController.class)
public class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @Test
    public void testGetUser() throws Exception {
        User user = new User();

        User user = new User();
        user.setId(1L);
        user.setName("Priya Sharma");

        when(userService.getUserById(1L)).thenReturn(user);

        mockMvc.perform(get("/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("Priya Sharma"));
    }
}

```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 4: Integration Test with Spring Boot

UserIntegrationTest.java

```
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.web.client.TestRestTemplate;
import org.springframework.boot.test.web.server.LocalServerPort;
import org.springframework.http.ResponseEntity;

import static org.junit.jupiter.api.Assertions.*;

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class UserIntegrationTest {

    @LocalServerPort
    private int port;

    @Autowired
    private TestRestTemplate restTemplate;

    @Autowired
    private UserRepository userRepository;

    @Test
    @Test
    public void testGetUserEndToEnd() {
        User user = new User();
        user.setName("Arjun Mehta");
        User saved = userRepository.save(user);

        ResponseEntity<User> response = restTemplate.getForEntity(
            "/users/" + saved.getId(), User.class);

        assertEquals(200, response.getStatusCodeValue());
        assertEquals("Arjun Mehta", response.getBody().getName());
    }
}
```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 5: Test Controller POST Endpoint

UserControllerPostTest.java

```

import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.Mockito.*;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(UserController.class)
public class UserControllerPostTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @Test
    public void testCreateUser() throws Exception {
        User inputUser = new User();
        inputUser.setName("Sneha Reddy");

        User savedUser = new User();
        savedUser.setId(1L);
        savedUser.setName("Sneha Reddy");

        when(userService.saveUser(any(User.class))).thenReturn(savedUser);

        ObjectMapper mapper = new ObjectMapper();

        mockMvc.perform(post("/users")
            .contentType(MediaType.APPLICATION_JSON)
            .content(mapper.writeValueAsString(inputUser)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value(1))
            .andExpect(jsonPath("$.name").value("Sneha Reddy"));
    }
}

```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 6: Test Service Exception Handling

UserServiceExceptionTest.java

```

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
public class UserServiceExceptionTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    @Test
    public void testGetUserByIdReturnsNullWhenMissing() {
        when(userRepository.findById(99L)).thenReturn(Optional.empty());

        User result = userService.getUserById(99L);

        assertNull(result);
    }
}

```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 7: Test Custom Repository Query

UserRepositoryTest.java

```

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;

import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

@DataJpaTest
public class UserRepositoryTest {

    @Autowired
    private UserRepository userRepository;

    @Test
    public void testFindByName() {
        User user1 = new User();
        user1.setName("Kiran Rao");
        userRepository.save(user1);

        User user2 = new User();
        user2.setName("Kiran Rao");
        userRepository.save(user2);
    }
}

```

```

        List<User> results = userRepository.findByName("Kiran Rao");

        assertEquals(2, results.size());
    }
}

```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 8: Test Controller Exception Handling

GlobalExceptionHandlerTest.java

```

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.servlet.MockMvc;

import java.util.NoSuchElementException;
import static org.mockito.Mockito.*;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(UserController.class)
public class GlobalExceptionHandlerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @Test
    public void testUserNotFoundReturns404() throws Exception {
        when(userService.getUserById(5L)).thenReturn(new NoSuchElementException());

        mockMvc.perform(get("/users/5"))
            .andExpect(status().isNotFound())
            .andExpect(content().string("User not found"));
    }
}

```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 9: Parameterized Test with JUnit

CalculatorParameterizedTest.java

```

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import static org.junit.jupiter.api.Assertions.*;

public class CalculatorParameterizedTest {

    CalculatorService calculatorService = new CalculatorService();

    @ParameterizedTest
    @CsvSource({
        "2, 3, 5",
        "10, 5, 15",
        "-1, 1, 0",
        "0, 0, 0"
    })
    public void testAddWithMultipleInputs(int a, int b, int expected) {
        assertEquals(expected, calculatorService.add(a, b));
    }
}

```

Output:

```

Running VoidExceptionTest          - 1 passed
Running CalculatorParameterizedTest - 4 passed (2+3=5, 10+5=15, -1+1=0, 0+0=0)
Running CalculatorTest              - 3 passed
Running SetupCheckTest             - 1 passed

Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

```